物联网学习计划（第二版）

> **目标用户**：PYJTLK
> **身份**：汕头大学机械专业校企联培研究生，研二下学期
> **目标**：走物联网方向，与智慧农业结合，达到中大型企业应聘要求
> **计划周期**：2026年5月1日 – 2027年6月（研三下学期毕业）

一、整体架构总览

1.1 用户能力现状分析

能力维度	当前水平	本计划处理方式
硬件/MCU	STM32、ESP32、Arduino、I2C/SPI	✅ 已掌握，快速回顾即可
机器人/AI	ROS2、YOLOv5、PX4无人机	✅ 已掌握，作为差异化优势
编程语言	C/C++/Java/Python	✅ 有基础，快速上手新语言
物联网协议	MQTT 3.0/5.0已学，LoRa仅用串口	⚠️ 需补充CoAP、LoRa协议层
网页开发	完全不会	❌ **最大缺口，重点投入**
容器化	Docker未接触	❌ 需系统学习
前端框架	无经验	❌ 需从零学习

1.2 技术栈决策

```
┌──────────────────────────────────────────────────────────────┐
│                          最终技术栈                          │
├──────────────────────────────────────────────────────────────┤
│  后端：Python FastAPI          （自研项目使用）              │
│  前端：Vue3 + TypeScript       （自研项目使用）              │
│  协议：MQTT/CoAP/Http          （设备通信）                  │
│  数据库：PostgreSQL + Redis    （业务数据）                  │
│  时序库：InfluxDB/TDengine     （传感器数据）                │
│  消息队列：RabbitMQ/Kafka      （异步处理）                  │
```

```
│  容器: Docker + Docker Compose (部署)                          │
│  书中学Node.js: 仅跟书学习，能看懂跑通即可                      │
└──────────────────────────────────────────────────────────────┘
```

1.3 阶段划分总览

```
时间轴: 2026年5月 ──────────────────────────────────────────► 2027年6月
        │                                                        │
        ▼                                                        ▼

┌──────────────┐   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐
│ 阶段一       │   │ 阶段二       │   │ 阶段三       │   │ 阶段四       │
│ Web开发基础  │   │ IoT核心深入  │   │ 项目实战与   │   │ 求职准备     │
│ (JS+Node.js+ │   │ (协议+平台+  │   │ 综合项目     │   │ +面试冲刺    │
│  Vue3+FastAPI)│   │  Docker)     │   │              │   │              │
├──────────────┤   ├──────────────┤   ├──────────────┤   ├──────────────┤
│ 约3个月      │   │ 约3个月      │   │ 约8个月      │   │ 约3个月      │
│ 5月-7月      │   │ 8月-10月     │   │ 11月-6月     │   │ 4月-6月      │
└──────────────┘   └──────────────┘   └──────────────┘   └──────────────┘
```

二、阶段一：Web开发基础（2026年5月 - 2026年7月）

2.1 阶段目标

- 掌握JavaScript基础语法，能阅读和编写简单JS代码
- 理解Node.js核心概念，能跑通《物联网系统开发》书中的Node.js代码
- 掌握Vue3基础用法，能构建简单的数据展示页面
- 了解FastAPI基础，能搭建简单的REST API

2.2 每周任务分解

第1-2周：JavaScript基础（跟《完美图解物联网IoT实操》）

日期	任务内容	产出物
周一(2.5h)	JS变量、数据类型、函数基础	练习代码3-5个
周二(2.5h)	条件判断、循环、数组操作	练习代码3-5个
周三(2.5h)	对象、DOM基础、事件处理	完成简单网页交互
周四(2.5h)	异步编程：Promise/async/await	理解回调与异步
周五(2.5h)	JSON格式、fetch API基础	能发起HTTP请求
周六(6h)	**综合练习**：用JS实现温湿度数据展示	前端页面1个
周日(6h)	**综合练习**：用JS实现MQTT模拟收发	小项目1个

> **学习资源**：
> - 《完美图解物联网IoT实操》第1章 JavaScript入门
> - MDN JavaScript教程：https://developer.mozilla.org/zh-CN/docs/Learn/JavaScript

第3-4周：Node.js入门（跟《物联网系统开发》第7章）

日期	任务内容	产出物
周一(2.5h)	Node.js环境安装、REPL使用	环境验证成功
周二(2.5h)	核心模块：fs/http/path/os	理解模块化
周三(2.5h)	npm包管理、package.json	会安装依赖
周四(2.5h)	Express框架入门	跑通第一个服务器
周五(2.5h)	路由、请求处理、中间件	理解Express逻辑
周六(6h)	**跟书实践**：搭建Maque IotHub准备工作台	项目环境就绪
周日(6h)	**跟书实践**：运行第7章完整示例代码	能跑通示例

> **学习目标**：Node.js不需要精通，只需能看懂书中Node.js代码逻辑即可

第5-6周：Vue3基础

日期	任务内容	产出物
周一(2.5h)	Vite + Vue3项目创建	项目结构理解
周二(2.5h)	Composition API：ref/reactive/computed	响应式基础
周三(2.5h)	生命周期、watch/watchEffect	组件生命周期
周四(2.5h)	Props/Emit/组件通信	组件拆分能力
周五(2.5h)	插槽、依赖注入	高级组件模式
周六(6h)	**实战**：用Vue3实现设备列表页面	组件2-3个
周日(6h)	**实战**：用Vue3实现实时数据展示	页面1个

> **学习资源**：Vue3官方文档 + B站《Vue3零基础入门》

第7-8周：FastAPI后端入门

日期	任务内容	产出物
周一(2.5h)	FastAPI安装、项目结构	环境就绪
周二(2.5h)	路由、查询参数、请求体	REST API基础
周三(2.5h)	Pydantic数据验证	数据模型定义
周四(2.5h)	数据库ORM：SQLAlchemy	数据库操作
周五(2.5h)	认证：JWT Token	登录认证逻辑
周六(6h)	**实战**：FastAPI + Vue3实现设备管理API	完整CRUD
周日(6h)	**实战**：前后端联调、数据展示	完整小项目

第9-10周：阶段整合 + 跟书实战

日期	任务内容	产出物
周一(2.5h)	跟书第8章：设备生命周期管理	Node.js代码跟读
周二(2.5h)	跟书第9章：上行数据处理	理解数据流向
周三(2.5h)	跟书第10章：下行数据处理	RPC调用逻辑
周四(2.5h)	阶段总结：画技术架构图	知识梳理
周五(2.5h)	查漏补缺	薄弱点强化
周六(6h)	**实战**：用FastAPI+Vue3复刻书中设备注册功能	代码对比学习
周日(6h)	**周回顾**：整理笔记、代码仓库规范化	Git仓库完善

2.3 阶段一检验方式

- ☐ 能独立创建一个Vue3项目并实现数据展示
- ☐ 能用FastAPI搭建一个完整的设备管理API
- ☐ 能阅读并理解《物联网系统开发》第7-10章的Node.js代码
- ☐ 完成GitHub仓库：至少3个小型项目代码

三、阶段二：IoT核心深入（2026年8月 - 2026年10月）

3.1 阶段目标

- 深入理解MQTT 5.0新特性，能在实际项目中使用
- 掌握CoAP协议，能在低功耗设备场景选用合适协议
- 理解LoRaWAN协议层，不再仅停留在串口使用
- 掌握Docker容器化部署，能将IoT平台容器化部署
- 能跟书完成Maque IotHub核心功能开发

3.2 每周任务分解

第11-12周：MQTT 5.0深入（跟书第3-5章）

日期	任务内容	产出物
周一(2.5h)	MQTT 5.0新特性：属性扩展、用户属性	理解新特性
周二(2.5h)	共享订阅、请求响应模式	高级特性
周三(2.5h)	EMQX 5.0配置与使用	Broker熟练使用
周四(2.5h)	跟书实践：EMQX认证配置	认证流程
周五(2.5h)	跟书实践：MQTT 5.0特性验证	测试Demo
周六(6h)	**实战**：EMQX + MQTT 5.0实现设备分组	项目代码
周日(6h)	**实战**：实现带用户属性的消息追踪	项目代码

> **注意**：用户已有MQTT 3.1.1基础，本阶段重点是5.0新特性和EMQX实战

第13-14周：CoAP协议 + 设备接入（跟书第13章）

日期	任务内容	产出物
周一(2.5h)	CoAP协议原理：消息模型/请求响应	理解协议
周二(2.5h)	CoAP与MQTT对比：何时选哪个	技术选型能力
周三(2.5h)	EMQX CoAP网关配置	网关部署
周四(2.5h)	跟书实践：CoAP设备接入IoT平台	集成实现
周五(2.5h)	CoAP observe机制、资源订阅	理解观察者模式
周六(6h)	**实战**：ESP32 CoAP客户端实现	嵌入式代码
周日(6h)	**实战**：CoAP + MQTT混合接入	项目代码

第15-16周：Docker容器化

日期	任务内容	产出物
周一(2.5h)	Docker基础：镜像/容器/仓库概念	理解核心概念
周二(2.5h)	Docker命令：run/ps/pull/build	熟练使用
周三(2.5h)	Dockerfile编写：指令详解	能写Dockerfile
周四(2.5h)	Docker网络、数据卷、持久化	网络配置
周五(2.5h)	Docker Compose多容器编排	集群管理
周六(6h)	**实战**：FastAPI + Redis + PostgreSQL容器化	docker-compose.yml
周日(6h)	**实战**：EMQX + InfluxDB + FastAPI完整部署	容器化架构

> **Docker学习深度**：掌握到能容器化部署自研项目即可，不需要精通K8s

第17-18周：LoRa协议深入 + 跟书高级功能（跟书第11章）

日期	任务内容	产出物
周一(2.5h)	LoRaWAN协议栈深入：MAC层/帧结构	理解协议层
周二(2.5h)	LoRa Class A/B/C 工作模式	模式选择
周三(2.5h)	跟书：NTP服务、设备分组	功能理解
周四(2.5h)	跟书：M2M设备间通信、设备影子	高级功能
周五(2.5h)	跟书：OTA升级实现原理	升级机制
周六(6h)	**实战**：模拟实现轻量级设备影子	项目代码
周日(6h)	**实战**：时序数据库InfluxDB集成	时序数据存储

第19-20周：阶段整合 + EMQX扩展

日期	任务内容	产出物
周一(2.5h)	跟书第12章：Erlang基础（够理解EMQX即可）	理解EMQX
周二(2.5h)	EMQX Hook机制、ExHook扩展	扩展开发
周三(2.5h)	RabbitMQ消息队列集成	消息中间件
周四(2.5h)	跟书第14章：gRPC钩子	微服务扩展
周五(2.5h)	阶段总结：画完整IoT架构图	知识梳理
周六(6h)	**实战**：用Docker Compose搭建完整IoT平台	一键部署
周日(6h)	**周回顾**：整理笔记、完善项目文档	文档产出

3.3 阶段二检验方式

- ☐ 能使用MQTT 5.0新特性进行项目开发
- ☐ 能为ESP32等设备编写CoAP客户端代码
- ☐ 能用Docker一键部署IoT平台所有组件
- ☐ 完成GitHub仓库：IoT平台容器化部署完整代码

四、阶段三：项目实战与综合能力（2026年11月 - 2027年6月）

4.1 阶段目标

- 完成自研IoT平台项目（FastAPI + Vue3 + MQTT）
- 完成智慧农业项目（融合YOLOv5/ROS2差异化技能）
- 积累完整项目经验，可作为求职作品集
- 达到中大型企业物联网岗位技术要求

4.2 项目一：自研IoT平台（11月 - 2月）

4.2.1 项目概述

```
┌──────────────────────────────────────────────────────────────┐
│                      自研IoT平台架构图                        │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│    ┌──────────┐      ┌──────────┐      ┌──────────┐         │
│    │ ESP32    │      │ STM32    │      │ LoRa     │         │
│    │ 设备     │      │ 设备     │      │ 节点     │         │
│    └──────────┘      └──────────┘      └──────────┘         │
│         │                 │                 │               │
│         └─────────────────┴─────────────────┘               │
│                           ▼                                 │
│                  ┌──────────────────┐                       │
│                  │   EMQX Broker    │                       │
│                  │   (Docker)       │                       │
│                  └──────────────────┘                       │
│                           │                                 │
│          ┌────────────────┴────────────────┐                │
│          ▼                ▼                ▼                │
│   ┌──────────┐    ┌──────────┐    ┌──────────┐             │
│   │ FastAPI  │    │ InfluxDB │    │ RabbitMQ │             │
│   │ 后端API  │    │ 时序数据 │    │ 消息队列 │             │
│   └──────────┘    └──────────┘    └──────────┘             │
│        │                                                    │
│        ▼                                                    │
│   ┌──────────────────────────────────┐                      │
│   │          Vue3 前端               │                      │
│   │  ┌────────┐ ┌────────┐ ┌──────┐ │                      │
│   │  │设备管理│ │数据看板│ │告警  │ │                      │
│   │  └────────┘ └────────┘ └──────┘ │                      │
│   └──────────────────────────────────┘                      │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

4.2.2 功能模块

模块	功能点	技术实现	预计周期
设备管理	设备注册/认证/分组/禁用	FastAPI + JWT + MQTT ACL	3周
数据采集	设备数据上报/存储/查询	InfluxDB + MQTT + FastAPI	3周
远程控制		MQTT RPC + 异步任务	2周

模块	功能点	技术实现	预计周期
	下行命令/RPC调用/OTA		
数据可视化	实时数据/历史曲线/报表	ECharts + Vue3	2周
告警管理	阈值告警/通知/记录	规则引擎 + RabbitMQ	2周
用户权限	多角色/多租户/API密钥	JWT + 数据库权限设计	2周

4.2.3 每周任务

周次	任务内容	产出物
第21周	项目初始化：Git仓库/Vue3项目/FastAPI项目/Docker环境	项目框架
第22周	设备模块：设备注册API + MQTT认证 + 设备列表前端	基础功能
第23周	设备模块：设备详情/分组/禁用功能 + 前端页面	完整功能
第24周	数据模块：InfluxDB集成 + MQTT数据订阅 + 存储实现	数据层
第25周	数据模块：数据查询API + 历史数据接口	API完成
第26周	可视化：实时数据展示 + ECharts集成	图表页面
第27周	可视化：历史曲线/数据导出/仪表盘	完整看板
第28周	控制模块：下行命令/RPC调用/命令历史	控制功能
第29周	告警模块：告警规则配置/阈值设置	规则引擎
第30周		通知系统

周次	任务内容	产出物
	告警模块：告警通知/微信/邮件 + 告警记录	
第31周	权限模块：JWT认证/角色管理/操作日志	安全加固
第32周	容器化部署：Dockerfile + docker-compose	一键部署
第33周	收尾优化：文档完善/Bug修复/性能优化	稳定版本
第34周	项目总结：架构文档/使用手册/演示视频	完整交付

4.3 项目二：智慧农业监控系统（2027年2月 - 5月）

4.3.1 项目定位（差异化竞争优势）

```
┌──────────────────────────────────────────────────────────────┐
│                      智慧农业项目差异化定位                    │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   传统智慧农业痛点：                                          │
│   ❌仅做数据采集和展示                                        │
│   ❌缺乏智能分析和决策                                        │
│   ❌与用户已有技能（YOLOv5/ROS2）脱节                         │
│                                                              │
│   本项目差异化优势：                                          │
│   ✅融合YOLOv5：病虫害AI识别                                  │
│   ✅融合ROS2：农业机器人/无人机协作                           │
│   ✅垂直农场场景：精准环境控制                                │
│   ✅预测性维护：设备状态预警                                  │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

4.3.2 项目架构

```
┌──────────────────────────────────────────────────────────────┐
│                      智慧农业监控系统架构                      │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   ┌──────────────────────────────────────────────────────┐   │
│   │                      用户交互层                       │   │
│   │  Vue3 Web管理后台  │  移动端小程序(可选)              │   │
│   └──────────────────────────────────────────────────────┘   │
│                              │                               │
│                              ▼                               │
│   ┌──────────────────────────────────────────────────────┐   │
│   │                      业务应用层                       │   │
│   │  ┌──────────┐ ┌──────────┐ ┌──────────┐              │   │
│   │  │环境监控  │ │AI视觉    │ │设备控制  │              │   │
│   │  │子系统    │ │子系统    │ │子系统    │              │   │
│   │  └──────────┘ └──────────┘ └──────────┘              │   │
│   └──────────────────────────────────────────────────────┘   │
│            │            │            │                       │
│            ▼            ▼            ▼                       │
│   ┌──────────────────────────────────────────────────────┐   │
│   │                      数据服务层                       │   │
│   │  ┌──────────┐ ┌──────────┐ ┌──────────┐              │   │
│   │  │时序数据库│ │AI模型库  │ │规则引擎  │              │   │
│   │  │InfluxDB  │ │YOLOv5    │ │RabbitMQ  │              │   │
│   │  └──────────┘ └──────────┘ └──────────┘              │   │
│   └──────────────────────────────────────────────────────┘   │
│                              │                               │
│                              ▼                               │
│   ┌──────────────────────────────────────────────────────┐   │
│   │                      设备接入层                       │   │
│   │  ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐  │   │
│   │  │环境传感器│ │电磁阀    │ │YOLOv5    │ │ROS2      │  │   │
│   │  │(ESP32)   │ │(继电器)  │ │摄像头    │ │机器人    │  │   │
│   │  └──────────┘ └──────────┘ └──────────┘ └──────────┘  │   │
│   └──────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────┘
```

4.3.3 核心功能模块

模块	功能描述	技术实现	差异化点
环境监控	温湿度/光照/CO2/土壤墒情	ESP32 + MQTT	多协议支持
AI病虫害识别		Python + OpenCV	**用户独特技能**

模块	功能描述	技术实现	差异化点
	摄像头实时监控 + YOLOv5识别		
智能灌溉	根据土壤墒情 + 天气预测自动灌溉	FastAPI + 继电器	预测性决策
垂直农场控制	层架式种植 + LED补光 + 通风控制	ROS2节点	**用户独特技能**
无人机巡检	ROS2无人机定期巡检 + 热成像	PX4 + MAVROS	**用户独特技能**
数据分析	产量预测/病虫害趋势/环境优化建议	机器学习	智能分析

4.3.4 每周任务

周次	任务内容	产出物
第35周	项目规划：需求分析/技术选型/架构设计	设计文档
第36周	基础环境：硬件连线/固件开发/测试环境	设备就绪
第37周	环境监控：传感器数据采集/MQTT上报/存储	数据链路
第38周	环境监控：Web展示/告警规则	监控面板
第39周	**AI模块**：YOLOv5模型训练/部署	模型可用
第40周	**AI模块**：实时识别API/告警联动	智能识别
第41周	灌溉控制：自动灌溉逻辑/手动控制	控制功能
第42周	**ROS2集成**：机器人节点/任务调度	机器人协作
第43周		巡检功能

周次	任务内容	产出物
	无人机集成：巡检航线/数据回传	
第44周	数据分析：产量预测模型/报表生成	数据分析
第45周	完整集成测试/性能优化	系统稳定
第46周	文档完善：用户手册/部署文档/演示视频	完整交付

4.4 阶段三检验方式

- ☐ 自研IoT平台：完整功能、可演示、可部署
- ☐ 智慧农业项目：包含YOLOv5/ROS2差异化亮点
- ☐ GitHub仓库：两个完整项目 + README + 架构文档
- ☐ 作品集网站：可在线演示（可选）

五、阶段四：求职准备（2027年4月 - 2027年6月）

5.1 阶段目标

- 完善简历，突出物联网技能和项目经验
- 准备面试，刷题+项目深挖
- 投递简历，获得心仪offer

5.2 简历优化

5.2.1 技术栈梳理

```
┌──────────────────────────────────────────────────────────┐
│                          简历技术栈亮点                          │
├──────────────────────────────────────────────────────────┤
│                                                                  │
│   【物联网平台开发】                                              │
```

```
│   • 熟练使用MQTT/CoAP协议进行设备接入                                  │
│   • 掌握EMQX消息中间件的部署与扩展开发                                 │
│   • 熟悉Docker容器化部署，有完整IoT平台项目经验                        │
│                                                                        │
│   【后端开发】                                                         │
│   • 熟练使用Python FastAPI构建REST API                                 │
│   • 掌握PostgreSQL/Redis/InfluxDB数据库使用                            │
│   • 熟悉JWT认证、权限管理、API设计规范                                 │
│                                                                        │
│   【前端开发】                                                         │
│   • 熟练使用Vue3 + Composition API进行前端开发                         │
│   • 掌握ECharts数据可视化、响应式页面设计                              │
│                                                                        │
│   【嵌入式开发】（加分项）                                             │
│   • 熟悉STM32/ESP32单片机开发                                          │
│   • 有ROS2机器人开发经验、YOLOv5目标检测部署经验                       │
│                                                                        │
│   【智慧农业方向】（差异化优势）                                       │
│   • 熟悉智慧农业IoT系统架构设计                                        │
│   • 融合AI视觉(病虫害识别)、机器人协作、无人机巡检                     │
│                                                                        │
└────────────────────────────────────────────────────────────────────────┘
```

5.2.2 项目描述模板

```
【项目一：自研IoT平台】- FastAPI + Vue3 + MQTT
项目描述：独立开发了一个支持多协议设备接入的物联网平台，实现设备管理、数据采集、
    远程控制、数据可视化等核心功能，支持MQTT/CoAP协议，Docker容器化部署。
技术栈：Python FastAPI / Vue3 / EMQX / InfluxDB / Docker
个人职责：架构设计 / 后端开发 / 前端开发 / 部署运维
项目成果：支持100+设备稳定接入，数据延迟<100ms

【项目二：智慧农业监控系统】- YOLOv5 + ROS2 + MQTT
项目描述：针对垂直农场场景开发的智能监控系统，融合AI视觉识别、机器人协作、
    无人机巡检，实现病虫害实时检测、环境智能调控。
技术栈：Python / YOLOv5 / ROS2 / ESP32 / FastAPI / Vue3
个人职责：系统架构 / AI模型集成 / 设备控制逻辑 / ROS2节点开发
项目成果：病虫害识别准确率>90%，减少人工巡检80%
```

5.3 面试准备

5.3.1 技术知识清单

类别	必问问题	准备要点
MQTT	QoS机制/遗嘱消息/会话保持	能画图解释原理
CoAP	与MQTT区别/observe机制	理解适用场景
物联网架构	设备接入→数据处理→应用	能画出完整架构
FastAPI	异步/依赖注入/中间件	理解设计理念
Vue3	Composition API/响应式原理	理解设计思想
Docker	镜像层/网络模式/数据卷	熟练使用命令
数据库	关系型vs时序库/索引优化	理解选型原因
智慧农业	场景理解/技术选型理由	有独到见解

5.3.2 每周复习计划

周次	任务	内容
第47周	MQTT/CoAP深入	协议原理、源码级理解
第48周	FastAPI/Vue3深入	框架设计思想、源码
第49周	Docker/数据库深入	原理、实战问题
第50周	项目深挖	架构决策、难点解决
第51周	模拟面试	自我介绍、项目讲解
第52周	简历投递	目标公司、岗位分析

5.4 阶段四检验方式

- ☐ 简历完善，突出物联网技能
- ☐ 能流畅讲解两个完整项目
- ☐ 技术知识清单全部覆盖
- ☐ 获得至少3个面试机会

六、关键决策说明

6.1 Node.js学习策略

```
┌──────────────────────────────────────────────────────────────┐
│                        Node.js定位说明                        │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  目标：跟《物联网系统开发》学习，能看懂、能跑通书中代码          │
│                                                              │
│  学习深度：                                                   │
│  ✓ 能安装Node.js环境和npm依赖                                 │
│  ✓ 能阅读和理解Express框架代码                                 │
│  ✓ 能跑通书中Maque IotHub的Node.js服务端代码                  │
│  ✓ 理解Node.js事件驱动、异步编程模型                           │
│                                                              │
│  不需要深入：                                                 │
│  ✗ 不需要精通Node.js后端开发                                   │
│  ✗ 不需要自己用Node.js开发完整项目                             │
│  ✗ 不需要深入Express中间件生态                                 │
│                                                              │
│  原因：用户选择FastAPI作为后端技术栈，Node.js仅用于跟书学习     │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

6.2 FastAPI学习重点

```
┌──────────────────────────────────────────────────────────────┐
│                        FastAPI学习重点                        │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  核心重点（必须掌握）：                                        │
│  ✓ 路由定义、查询参数、请求体、路径参数                         │
│  ✓ Pydantic数据验证、模型定义                                  │
│  ✓ 依赖注入系统                                               │
│  ✓ 异步操作、后台任务                                          │
│  ✓ JWT认证、JWT令牌验证                                        │
│  ✓ 数据库ORM（SQLAlchemy/Alembic）                             │
│  ✓ WebSocket实时通信                                           │
│  ✓ Docker部署                                                 │
│                                                              │
```

```
│  了解即可:                                                    │
│  • OpenAPI自动文档（会用即可）                                 │
│  • 测试框架（了解基本用法）                                    │
│  • 中间件开发（了解概念）                                      │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

6.3 Vue3学习重点

```
┌──────────────────────────────────────────────────────────────┐
│                        Vue3学习重点                          │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  核心重点（必须掌握）:                                         │
│  ✓ Composition API: ref/reactive/computed/watch              │
│  ✓组件通信: props/emit/provide/inject                         │
│  ✓生命周期钩子                                                 │
│  ✓ Vue Router: 路由配置、导航守卫                              │
│  ✓ Pinia状态管理                                               │
│  ✓ HTTP请求: axios封装                                         │
│  ✓ ECharts数据可视化集成                                       │
│  ✓ Vite工程化配置                                              │
│                                                              │
│  了解即可:                                                    │
│  • TypeScript集成（基础类型注解即可）                          │
│  • 单元测试（了解概念）                                        │
│  • SSR/Nuxt（了解概念）                                        │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

6.4 Docker学习范围

```
┌──────────────────────────────────────────────────────────────┐
│                        Docker学习范围                        │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  必须掌握:                                                    │
│  ✓ Docker安装和基本概念（镜像/容器/仓库）                      │
│  ✓常用命令: pull/run/ps/stop/rm/exec/logs                     │
│  ✓ Dockerfile编写: FROM/RUN/COPY/WORKDIR/ENV/CMD              │
│  ✓ Docker Compose: services/networks/volumes                 │
```

```
│   ✓ 镜像构建和推送                                                  │
│   ✓ 网络配置和数据卷                                                 │
│                                                                      │
│   了解概念即可:                                                      │
│   • Docker Swarm                                                     │
│   • Kubernetes（有时间再学）                                         │
│   • 镜像安全扫描                                                     │
│   • CI/CD集成                                                        │
│                                                                      │
│   学习深度: 以能容器化部署自研IoT平台为准                            │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

七、每日/每周时间安排

7.1 工作日安排（周一至周五）

```
┌──────────────────────────────────────────────────────────────────────┐
│                        工作日每日安排（2.5-3小时）                   │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   19:00 - 19:30   任务启动                                           │
│   │               • 查看当日任务                                     │
│   │               • 环境准备/代码拉取                                │
│   │               • 15分钟快速回顾昨日内容                           │
│   │                                                                  │
│   19:30 - 21:30   核心任务时间（约2小时）                            │
│   │               • 看书/看教程（40分钟）                            │
│   │               • 代码实践（60分钟）                               │
│   │               • 调试/问题解决（20分钟）                          │
│   │                                                                  │
│   21:30 - 21:45   收尾整理（15分钟）                                 │
│   │               • 提交代码到Git                                    │
│   │               • 记录当日进度                                     │
│   │               • 简单笔记/问题记录                                │
│   │                                                                  │
│   21:45 - 22:00   明日规划（可选）                                   │
│   │               • 列出明日任务清单                                 │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

7.2 周末安排（周六周日）

```
┌──────────────────────────────────────────────────────────────┐
│                      周末每日安排（6-7小时）                    │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  08:00 - 09:00   起床/早餐/准备                               │
│  │                                                           │
│  09:00 - 12:00   上午任务块（3小时）                          │
│  │               • 周六：跟书实践/大块学习                    │
│  │               • 周日：项目实战/综合练习                    │
│  │                                                           │
│  12:00 - 14:00   午餐/休息                                    │
│  │                                                           │
│  14:00 - 17:00   下午任务块（3小时）                          │
│  │               • 周六：跟书实战/问题复盘                    │
│  │               • 周日：项目开发/功能实现                    │
│  │                                                           │
│  17:00 - 18:00   收尾整理                                     │
│  │               • 代码提交                                   │
│  │               • 周报/周总结                                │
│  │               • 笔记整理                                   │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

7.3 每周节奏

```
┌──────────────────────────────────────────────────────────────┐
│                          每周节奏                            │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  周一 - 周五：                                                │
│  • 每日约2.5-3小时                                            │
│  • 跟书学习 + 基础练习                                        │
│  • 合计约13-15小时                                            │
│                                                              │
│  周六 - 周日：                                                │
│  • 每日约6-7小时                                              │
│  • 项目实战 + 综合练习                                        │
│  • 合计约12-14小时                                            │
│                                                              │
│  ──────────────────────────────────────────                  │
```

```
│                                                                  │
│  周总计：约25-29小时（目标26小时）                                 │
│                                                                  │
│  周日晚上：周回顾与下周规划                                        │
│  │  • 整理本周学习内容                                            │
│  │  • 完成Git提交和文档更新                                       │
│  │  • 评估进度，调整下周计划                                      │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

八、关键里程碑

8.1 时间节点

里程碑	时间	标志事件
阶段一完成	2026年7月底	掌握JS+Node.js+Vue3+FastAPI基础
阶段二完成	2026年10月底	掌握MQTT5.0/CoAP/Docker
IoT平台完成	2027年2月底	自研IoT平台完整可运行
智慧农业完成	2027年5月底	完整项目交付
求职准备完成	2027年6月	获得offer

8.2 阶段成果检查点

```
2026年5月 ──────► 7月 ──────► 10月 ──────► 2月 ──────► 5月 ──────► 6月
  │                │            │            │            │            │
  │          ┌─────────────┐    │      ┌─────────────┐    │      │
  │          │ 阶段一检验  │    │      │ IoT平台     │    │      │
  │          │ ✓Vue3项目   │    │      │ ✓完整功能   │    │      │
  │          │ ✓FastAPI API│    │      │ ✓可部署     │    │      │
  │          │ ✓跟书Node.js│    │      │ ✓可演示     │    │      │
  │          └─────────────┘    │      └─────────────┘    │      │
  │                             │                         │      │
  │                     ┌─────────────┐           ┌─────────────┐
  │                     │ 阶段二检验  │           │ 智慧农业项目 │
  │                     │ ✓MQTT 5.0   │           │ ✓AI识别     │
```

```
    │                        │ ✓CoAP协议   │              │ ✓ROS2集成    │
    │                        │ ✓Docker部署│              │ ✓完整交付    │
    │                        └────────────┘              └──────────────┘
    │
    │
    │                                                    ┌──────────────┐
    │                                                    │ 求职准备     │
    │                                                    │ ✓简历完善    │
    │                                                    │ ✓面试通过    │
    │                                                    │ ✓Offer获取   │
    │                                                    └──────────────┘
```

九、学习资源汇总

9.1 核心教材

书籍	用途	使用阶段
《物联网系统开发：从0到1构建IoT平台（第2版）》	IoT平台实战	阶段一后半+阶段二
《完美图解物联网IoT实操》	JS/Node.js入门	阶段一

9.2 在线资源

资源	用途
Vue3官方文档	Vue3学习
FastAPI官方文档	后端学习
MDN JavaScript教程	JS基础
Docker官方文档	容器化
EMQX官方文档	MQTT Broker

9.3 技术文档

技术	重点文档
MQTT	协议规范、EMQX配置

技术	重点文档
CoAP	RFC 7252、libcoap库文档
InfluxDB	官方文档、Telegraf配置
Docker	Dockerfile最佳实践

十、风险与调整

10.1 常见风险

风险	应对策略
工作日时间不足	优先保证核心任务，周末补充
Node.js理解困难	多做跟书练习，不深入原理
项目进度延迟	精简功能，保证核心可用
遗忘前面内容	定期回顾，每周末整理笔记

10.2 调整原则

```
1. 进度优先：完成比完美重要，每个阶段都要有可交付物
2. 核心优先：核心功能必须完成，扩展功能可后延
3. 实践优先：看书为辅，动手为主
4. 定期回顾：每周日回顾本周内容，避免遗忘
```

十一、附录

11.1 环境准备清单

```
┌──────────────────────────────────────────────────────────┐
│                        开发环境准备                        │
├──────────────────────────────────────────────────────────┤
```

```
│                                                                    │
│  软件环境:                                                          │
│  □ Node.js 18+                                                     │
│  □ Python 3.10+                                                    │
│  □ Docker Desktop / Docker Engine                                  │
│  □ Docker Compose                                                  │
│  □ Git                                                             │
│  □ VS Code                                                         │
│  □ PostgreSQL（本地或Docker）                                       │
│  □ Redis（本地或Docker）                                            │
│  □ InfluxDB（Docker）                                               │
│  □ EMQX（Docker）                                                   │
│  □ MQTTX（MQTT客户端工具）                                          │
│                                                                    │
│  硬件设备（可选）:                                                  │
│  □ ESP32开发板 × 1-2                                               │
│  □ DHT22温湿度传感器 × 1                                            │
│  □ 继电器模块 × 1                                                   │
│  □ 摄像头模块（可选）                                               │
│                                                                    │
└────────────────────────────────────────────────────────────────────┘
```

11.2 GitHub仓库规划

```
┌────────────────────────────────────────────────────────────────────┐
│                        GitHub仓库规划                              │
├────────────────────────────────────────────────────────────────────┤
│                                                                    │
│  仓库1: iot-learning                                                │
│  │   用途: 学习过程中的练习代码                                      │
│  │   结构:                                                          │
│  │   ├── js-basics/     # JavaScript练习                            │
│  │   ├── nodejs-iot/     # 跟书Node.js代码                          │
│  │   ├── vue3-quickstart/ # Vue3入门练习                            │
│  │   └── fastapi-demo/   # FastAPI练习                              │
│                                                                    │
│  仓库2: iot-platform                                                │
│  │   用途: 自研IoT平台项目                                           │
│  │   结构:                                                          │
│  │   ├── backend/       # FastAPI后端                               │
│  │   ├── frontend/      # Vue3前端                                  │
│  │   ├── docker/       # Docker配置                                 │
│  │   └── docs/         # 架构文档                                   │
│                                                                    │
```

```
│  仓库3: smart-agriculture                                    │
│  │   用途: 智慧农业监控项目                                    │
│  │   结构:                                                    │
│  │   ├── hardware/     # ESP32固件                            │
│  │   ├── backend/      # FastAPI后端                          │
│  │   ├── frontend/     # Vue3前端                             │
│  │   ├── ai/          # YOLOv5模型                            │
│  │   ├── ros2/        # ROS2节点                              │
│  │   └── docs/        # 项目文档                              │
│                                                               │
└───────────────────────────────────────────────────────────────┘
```

> **计划制定时间**：2026年4月
> **计划版本**：v2.0
> **下一步行动**：确认计划 → 环境准备 → 2026年5月1日开始执行